

Python 程序设计基础第三章作业

姓名：路柠泽

学号：2022010076

1. 习题 1

1.1 题目

给定自然数 n ，编写函数，求其各位数字之和，如数 1234 各位数字之和为 10。编写函数，重复上述过程，直至得到 1~9 之间的某个数。

1.2 题目思路

函数 1：创建 while 循环，通过对 10 取余数获得个位数字并累加，再利用对 10 整除前进一位，为 0 时循环停止。函数 2：创建 while 循环，重复用函数 1 处理自然数，结果为一位数时停止。提示用户输入自然数并输出函数运算结果。

1.3 实现代码

```
1  def sum(n):
2      s=0
3      while n>0:          #End when n is 0
4          s+=n%10          #Extract the units digit
5          n//=10           #Move forward
6      return s
7  def rsum(n):
8      while n>9:           #Repeat, end when n is single digit
9          n=sum(n)
10     return n
11  n=int(input("Input a natural number: "))    #Test
12  print(sum(n))
13  print(rsum(n))
```

1.4 本地测试结果

- Case1-用户输入 13579，输出结果分别为 25，7，正确。

```
Input a natural number: 13579
25
7
```

- Case2-用户输入 8888888，输出结果分别为 56，2，正确。

```
Input a natural number: 8888888
56
2
```

2. 习题 2

2.1 题目

继续上一题。编写函数，检查 1 ~ 99999 之间所有数，给出最终结果中 1 ~ 9 出现比例。

2.2 题目思路

创建长度为 9 的列表，各项初值为 0，用于统计结果。通过区间内的 for 循环，利用上一题中函数检查 1~99999 之间的所有数，结果对应的列表元素值加 1。最后通过区间内 for 循环输出列表各项占总和的比例。

2.3 实现代码

```
1 > def sum(n): ...
7 > def rsum(n): ...
11 def check():
12     list=[0,0,0,0,0,0,0,0,0,0]
13     for i in range(1,100000):
14         list[rsum(i)-1]+=1
15     #Statistical analysis of results
16     for i in range(9):
17         list[i]/=99999
18         print(list[i])
19     #Calculate proportion and output
20     check()
```

2.4 本地测试结果

输出结果如下，说明最终结果中 1~9 出现的比例相等。

[illegible]

3. 习题 3

3.1 题目

编写函数，使用递归方法求 $C(n, k)$ 。

3.2 题目思路

由数学知识， k 为 0 或等于 n 时组合值为 1，并有恒等式： $C_n^k = C_{n-1}^{k-1} + C_{n-1}^k$ 。据此创建函数，当参数 k 为 0 或 k 、 n 相等时返回 1，其余情况返回 $C_{n-1}^{k-1} + C_{n-1}^k$ 进行递归运算。接受用户输入自然数 n ， k 后输出结果。

3.3 实现代码

```
1 def comb(n,k):
2     if k==0 or k==n:    #Special cases
3         return 1
4     else:
5         return comb(n-1,k-1)+comb(n-1,k)
6         #Recursive through identity
7 n=int(input()) #Test
8 k=int(input())
9 print(comb(n,k))
```

3.4 本地测试结果

- Case1-用户输入 9，3，输出结果为 84，正确。

```
9
3
84
```

- Case2-用户输入 3，3，输出结果为 1，正确。

```
3
3
1
```

4. 习题 4

4.1 题目

编写函数，计算圆周率。存在圆心在直角坐标系原点且半径为 1 的圆及其外切正方形。为计算方便，仅考虑位于第一象限的四分之一正方形和四分之一圆。随机生成该四分之一正方形中一系列点，散布于四分之一圆内比例即为圆周率四分之一。散步点越多，结果越精确，耗时也越长。

4.2 题目思路

创建一个双参数函数，分别接受随机点数目和计算次数。创建区间 for 循环的嵌套，外层为计算次数，内层为取点。random 模块中的 random.random() 能随机生成 0 到 1 间的浮点数，使用两次即生成一随机点，通过计算与原点的距离判断其是否位于四分之一单位圆内，并统计其内点的数目。内层循环结束后输出散布于四分之一圆内比例的四倍，即理论上圆周率的值。提示用户输入参数后运行函数。

4.3 实现代码

```
1 import random
2 def pi(n,t):
3     for i in range(t):
4         c=0 #The number of points in circle
5         for j in range(n):
6             x=random.random()
7             y=random.random() #Create a point
8             if x*x+y*y<=1: #Check if in circle
9                 c+=1
10            print(4*c/n) #Calculate pi and output
11 n=int(input("Input the number of random points: ")) #Test
12 t=int(input("Input the times of calculation: "))
13 pi(n,t)
```

4.4 本地测试结果

- Case1-用户输入 10000， 5， 输出结果如下：

```
Input the number of random points: 10000
Input the times of calculation: 5
3.1352
3.1568
3.16
3.13
3.1424
```

- Case2-用户输入 100000000， 5， 输出结果如下：

```
Input the number of random points: 100000000
Input the times of calculation: 5
3.14134488
3.14138464
3.14134696
3.14160004
3.14150316
```